hikami: A Lightweight Hypervisor for Emulating RISC-V Extension Semantics with Sail-Driven Auto-Generation

Anonymous Author(s)

Abstract

The rapid expansion of RISC-V extension specifications frequently outpaces hardware availability. This lag severely bottlenecks the development and validation of software stacks, which ultimately hinders the feedback loop necessary for actual hardware implementations. To address this challenge, we propose a lightweight Type-1 hypervisor that leverages the RISC-V Hypervisor extension to emulate the semantics of unimplemented RISC-V extensions. By trapping and emulating only the target instructions and control and status register (CSR) accesses, our system allows guest software to run natively for all supported instructions. Furthermore, to guarantee the correctness of the emulation toolchain and minimize manual effort, we introduce a novel auto-generation framework. We automatically derive the instruction decoders and hypervisor module templates directly from Sail, the official formal specifications of the RISC-V ISA written in Sail, thereby eliminating manual implementation errors. We evaluated our approach on a real RISC-V hardware platform (Milk-V Megrez). Experimental results demonstrate that our system outperforms a widely-used full-system emulator (QEMU) in realistic workloads, achieving faster execution times when emulated instructions account for 0.1% or less of the total execution. Additionally, our hypervisor maintains 99.8% of native performance for non-target workloads and restricts interrupt latency to under 1 microsecond. This formally-supported virtualization approach provides a practical, high-performance foundation for validating software against emerging RISC-V extensions prior to silicon availability.

CCS Concepts: • **Hardware** → *Simulation and emulation.*

Keywords: Hypervisor, RISC-V, Software Emulation, Embedded Systems

1 Introduction

The RISC-V instruction set architecture (ISA) [2, 20] has experienced a rapid expansion in its extension specifications, including a massive influx of officially ratified extensions. However, the physical hardware implementation of these diverse extensions frequently lags behind their standardization. This delay creates a significant bottleneck in the software development lifecycle, as operating systems, compilers, and applications cannot be fully validated until the target silicon becomes available. Conversely, hardware vendors are often reluctant to commit to silicon manufacturing without a mature, pre-existing software toolchain. This interdependence creates a vicious cycle that ultimately hinders the adoption of new extensions.

This hardware-software gap is particularly critical in the embedded systems domain. Embedded applications—ranging from IoT edge devices to real-time control systems—rely heavily on tight integration between the processor and specific hardware peripherals. Conventionally, developers rely on full-system emulators, such as QEMU, to test software prior to silicon availability. While these tools are invaluable, they often abstract away complex peripheral interactions and struggle to provide the accurate timing characteristics required for evaluating real-time embedded workloads. Furthermore, in practice, vendor-provided OS images and drivers for emerging hardware are frequently tightly coupled to the physical silicon; for example, they often bypass hardware abstraction layers like Device Trees to access specific physical components (e.g., IOMMUs) directly [15]. Running such heavily hardware-dependent software on full-system emulators requires implementing extensive and accurate models for undocumented or proprietary peripherals, which is highly impractical. Consequently, there is a pressing need for a validation environment that preserves near-native execution and real-world peripheral access (i.e., the ability to natively execute uncooperative or strictly hardware-coupled drivers) while supporting emerging, unimplemented ISA extensions.

To address this challenge, we propose a lightweight Type-1 hypervisor designed to emulate the semantics of unimplemented RISC-V extensions on existing silicon. By leveraging the RISC-V hardware virtualization features (the RISC-V Hypervisor extension), our system traps and emulates only the specific instructions, control and status registers (CSRs), and exceptions introduced by the new extensions. Because the system operates directly on actual silicon, this allows guest software to execute natively for all supported instructions and utilize existing device drivers to directly access underlying physical hardware peripherals exactly as they would on bare metal, eliminating the need for complex peripheral emulation.

Furthermore, emulator implementations are notoriously prone to manual coding errors, which can compromise the

LCTES '26, Boulder, Colorado, United States
2026. ACM ISBN 978-x-xxxx-xxxx-x/26/06
https://doi.org/10.1145/xxxxxxx.xxxxxxx

validity of software testing. To ensure the reliability of our emulation framework, we introduce an auto-generation approach. By utilizing Sail [4], the formal semantics language used for the official RISC-V ISA specifications, our toolchain automatically generates the instruction decoders and hypervisor module templates, thereby bridging the gap between formal specifications and practical system implementation.

In this paper, we formalize our investigation through the following research questions:

RQ1: How can we construct a near-native, low-latency execution environment for unimplemented RISC-V extensions?

RQ2: How can we build this emulation environment reliably while minimizing manual implementation effort and potential errors?

RQ3: What are the fundamental capabilities, performance characteristics, and boundaries of hypervisor-based software emulation in realistic workloads?

To answer these questions, we implemented our hypervisor on a real RISC-V hardware platform (Milk-V Megrez) [25]. The primary contributions of this paper are as follows:

- We designed and implemented a lightweight Type-1 hypervisor completely from scratch that enables the early validation of embedded software against unimplemented RISC-V extensions with a trap latency of under 1 microsecond.
- We developed a novel auto-generation framework that derives robust instruction decoders directly from the formal specifications written in Sail, significantly enhancing the reliability of the emulation toolchain.
- We demonstrated through empirical evaluation that our approach outperforms a widely-used full-system emulator (QEMU) in realistic scenarios—specifically, achieving faster execution times when emulated instructions account for 0.1% or less of the total execution. Furthermore, it maintains 99.8% of native performance for standard workloads that do not utilize the target extensions, ensuring that the hypervisor introduces negligible overhead to the guest OS and legacy applications.

2 Background and Related Work

2.1 RISC-V Extensions and the Hardware Gap

The RISC-V ISA is fundamentally designed around a modular architecture, consisting of a minimal base integer instruction set and numerous optional extensions [2]. While this modularity fosters innovation, it naturally results in a vast and continuously growing ecosystem of specifications. As of recent profiles [22], approximately 150 standard extensions have been officially ratified [23**?**]. Table 1 categorizes these ratified standard extensions by their architectural designators, highlighting the counts of newly introduced behaviors.

Table 1. Number of officially ratified standard ISA extensions categorized by their prefix. The single-letter category (A–Y) includes M, A, F, D, Q, C, B, P, V, and H, excluding the 'G' alias.

Prefix	Extension Category	Count
A–Y	Single-letter extensions	10
Z-	Standard unprivileged	82
Sm-	Machine mode	13
Ss-	Supervisor mode	17
Sv-	Virtual memory	12
Su-	User mode	1

Although the standardization process moves swiftly, building these diverse extensions into physical hardware takes considerable time, resulting in a significant gap between specifications and available silicon. To quantify this hardware gap, we conducted an empirical survey of 96 commercially available RISC-V hardware implementations listed by RISC-V International as of December 2025 [19]. Our analysis (summarized in Table 2) revealed that the vast majority of processors are limited to the RV32GC or RV64GC ISAs (where the 'G' alias encompasses IMAFDZicsr_Zifencei).

Only a small fraction of the surveyed hardware supports additional extensions such as Vector (V), Bit-Manipulation (B), Packed-SIMD (P), or the comprehensive RVA22 profile [21]. Most alarmingly, out of the approximately 150 ratified standard extensions, nearly 70% have no confirmed adoption in the surveyed commodity hardware. This severe adoption gap poses a significant threat to the health of the RISC-V ecosystem; it not only prevents developers from validating software stacks early but also hinders the retrofitting of newly ratified features onto existing deployed systems.

To bridge this gap, hardware-assisted approaches utilizing soft cores (e.g., Rocket Chip [5], VexRiscv [27], and CVA6 [30]) synthesized on FPGAs have been explored. Specifically, the Rocket Custom Coprocessor (RoCC) interface allows extending the processor with external accelerators for custom instructions, as demonstrated in domain-specific accelerations like SHA-3 [24], variable-precision floating-point operations [9], and double-precision vector operations [16].

While these approaches achieve near-native performance, the RoCC interface is designed explicitly for custom instructions and is inherently difficult to apply directly to the emulation of standard extensions. Furthermore, these hardware-centric methods require the entire system to be deployed on an FPGA. This reliance on FPGA synthesis or custom silicon tape-outs is not only costly and time-consuming but also introduces a significant hardware gap between the validation environment and the actual commercial off-the-shelf boards, severely limiting its utility in the early stages of software validation.

Table 2. Number of hardware implementations supporting each RISC-V extension out of 96 surveyed devices [19]. Extensions are grouped by G (left), RVA22 profile (center), and others (right).

Extension	Count
M	74
A	66
F	51
D	45
Zicsr	47
Zifencei	46

Extension	Count
C	72
B	14
Zba	2
Zbb	2
Zbs	2
V	22
Zicntr	8
Ziccif	8
Ziccrse	8
Ziccamoa	8
Zicclsm	8
Za64rs	8
Zihpm	9
Zihintpause	8
Zic64b	8
Zicbom	9
Zicbop	8
Zicboz	8
Zfhmin	8
Zkt	8

Extension	Count
P	9
K	2
Zbc	1
Zbc	1
Zbkb	1
Zbcb	1
Zbcmp	1

2.2 Software-Based Emulation Approaches

Given the hardware limitations, software-based emulation is widely employed. Full-system emulators like QEMU [8] are the de facto standards for pre-silicon software development. To address peripheral testing needs, recent works have developed sophisticated Virtual Prototypes (VPs). For instance, Pieper et al. proposed an open-source RISC-V VP enabling developers to manually model external physical components using scripting languages [18].

While highly versatile, these purely software-based environments inevitably abstract away complex timing characteristics and low-level interactions. More importantly, testing deeply hardware-coupled workloads on such platforms requires developers to accurately reimplement extensive software models for undocumented or proprietary physical peripherals, which is highly impractical. This fundamental limitation underscores the necessity for our hardware-assisted approach, which inherently preserves real-world peripheral access without requiring any software-based device modeling.

Another prominent approach is firmware-level emulation. For instance, customized projects such as OpenSBI-H [13] and vendor-specific modifications [29] have altered the trap vectors of OpenSBI to emulate missing extensions in Machine mode (M-mode) via illegal instruction traps. While transparent to the operating system and applicable to widely used single-board computers, executing complex emulation logic in M-mode—the highest privilege level—violates the principle of least privilege, posing significant security risks. Furthermore, M-mode lacks specific hardware virtualization support, leading to considerable trap-and-return overhead.

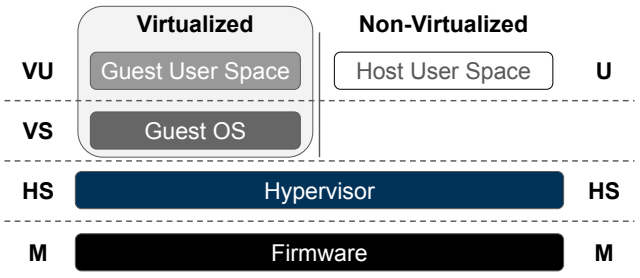

Figure 1. RISC-V privilege levels with the Hypervisor extension. Our Type-1 hypervisor operates in HS-mode, isolating emulation logic from M-mode.

Other approaches attempt emulation at the Supervisor (S-mode) or User (U-mode) levels. For example, projects like RVirt [6] execute both the guest OS and applications in U-mode, trapping all privileged instructions to an S-mode hypervisor. While this strictly limits privileges and enhances security, it incurs severe performance penalties due to the massive number of necessary traps and the high overhead of context switching between modes without hardware assistance.

In contrast, our approach utilizes a Type-1 hypervisor operating in Hypervisor-Extended Supervisor mode (HS-mode) [28]. While existing Type-1 hypervisors for embedded systems, such as Bao [17], primarily focus on static partitioning and hardware isolation for mixed-criticality workloads, our hypervisor is uniquely specialized and heavily optimized for the fast trap-and-emulate of unimplemented instructions. As shown in Figure 1, this architecture provides a balanced middle ground: it isolates the emulation logic from both the highly privileged M-mode and the guest OS, maintaining robust security. Crucially, executing in HS-mode allows our system to leverage hardware virtualization features, enabling significantly faster emulation with much lower overhead compared to M-mode or U-mode approaches, making it highly suitable for realistic embedded workloads.

2.3 RISC-V Virtualization (Hypervisor Extension)

The RISC-V hypervisor extension [3] introduces features to efficiently virtualize the processor. It alters the privilege architecture by adding Virtual Supervisor (VS) and Virtual User (VU) modes for guest execution, while the hypervisor runs in HS-mode. The hypervisor extension provides mechanisms such as two-stage address translation and hypervisor-specific Control and Status Registers (CSRs).

Crucially, the hypervisor extension provides robust hardware acceleration for trap handling, which is essential for

low-latency emulation. When an exception is delegated to HS-mode, the `htval` register automatically captures the faulting guest physical address, and the `htinst` register provides information about the faulting instruction itself. This hardware assistance eliminates the need for the hypervisor to perform expensive manual page table walks or fetch instructions from guest memory. Furthermore, special hypervisor virtual-machine load/store instructions (e.g., `hlv`, `hsv`) allow HS-mode to perform memory accesses using the guest's two-stage address translation seamlessly. Our system heavily relies on these hardware mechanisms to trap unallocated extension instructions and emulate their semantics with minimal latency.

2.4 Formal Semantics and Automated Generation

Developing emulators manually is an error-prone process, often leading to subtle discrepancies between the emulator behavior and the actual ISA specification. Historically, architecture specifications were defined using prose or pseudocode. These formats are neither rigorous nor executable, making them insufficient as a foundation for robust software development and verification.

To address this limitation, Armstrong et al. [4] proposed Sail, a formal semantics language specifically designed to rigorously describe ISAs. A defining feature of Sail is its capability to automatically generate multiple distinct targets from a single, unambiguous specification. For instance, it can generate fast standalone emulators written in C or OCaml, ISA tests, Verilog reference implementations, architectural documentation, and definitions for theorem provers such as Isabelle/HOL and Coq. Crucially, the formal model written in Sail is officially adopted in the RISC-V specification process as the formal specification of the architecture. Consequently, a vast repository of existing and emerging extensions is already formally described and publicly available [1]. Our system directly leverages these existing formal assets, requiring minimal additional effort from developers to support new specifications.

Furthermore, the official Sail model is continuously evolving. Recently, Deferme and Devriese [12] successfully formalized the RISC-V Hypervisor extension itself within the Sail model. Our research synergizes with this ongoing advancement of formal specifications by utilizing them to systematically elevate the reliability of the software virtualization layer.

Recent works, such as Pydrofoil [10], have demonstrated the utility of Sail by generating fast Instruction Set Simulators (ISS) directly from formal specifications. While Pydrofoil focuses on optimizing standalone simulators, our research applies Sail-driven automated generation directly to the domain of system-level virtualization. By automatically deriving instruction decoders and hypervisor module templates from the existing Sail specifications, we eliminate manual implementation errors and provide a highly reliable, formally-backed emulation framework suitable for kernel and driver testing.

3 Architecture and Emulation Framework

To address the challenges outlined in the previous sections, we designed and implemented a lightweight Type-1 hypervisor, customized specifically for validating unimplemented RISC-V extensions. This section details the system's overall architecture, its low-latency trap-and-emulate mechanisms, and the novel auto-generation framework backed by formal semantics.

3.1 System Overview

Our proposed system provides an end-to-end framework for emulating unimplemented RISC-V extensions, consisting of two primary components: an offline auto-generation toolchain and a runtime Type-1 hypervisor. As illustrated in the comprehensive system architecture in Figure 2, this design tightly integrates formal ISA specifications with a highly optimized execution environment.

To eliminate manual transcription errors and guarantee adherence to the RISC-V specifications, the offline toolchain takes the formal Sail semantics as input. It automatically derives precise instruction decoders, unit tests, and hypervisor module templates. Developers are only required to implement the core semantic logic within these generated templates, significantly reducing the development burden. The detailed generation pipeline will be discussed in Section 3.3.

At runtime, the system operates as a Type-1 hypervisor in the RISC-V Hypervisor-Extended Supervisor mode (HS-mode). Unlike general-purpose hypervisors designed primarily for multiplexing physical hardware among multiple virtual machines, our hypervisor boots directly on the bare-metal hardware and is heavily optimized for a single primary goal: intercepting and emulating unallocated extension instructions with minimal overhead. It loads the guest software (e.g., a guest OS or an RTOS) into Virtual Supervisor (VS) mode, allowing standard, hardware-supported instructions to execute natively.

To ensure maintainability and high cohesion, the emulation logic for each RISC-V extension is encapsulated within isolated hypervisor modules. Leveraging the Rust programming language's Cargo ecosystem, each extension module is packaged as an independent crate. This architecture provides exceptional modularity; developers can dynamically enable or disable support for specific extensions with only a few lines of configuration in the `Cargo.toml` file. When the guest software attempts to utilize an unimplemented feature, the hardware traps into the hypervisor, which identifies the appropriate module, routes the request, and emulates the exact architectural behavior before returning control to the guest.

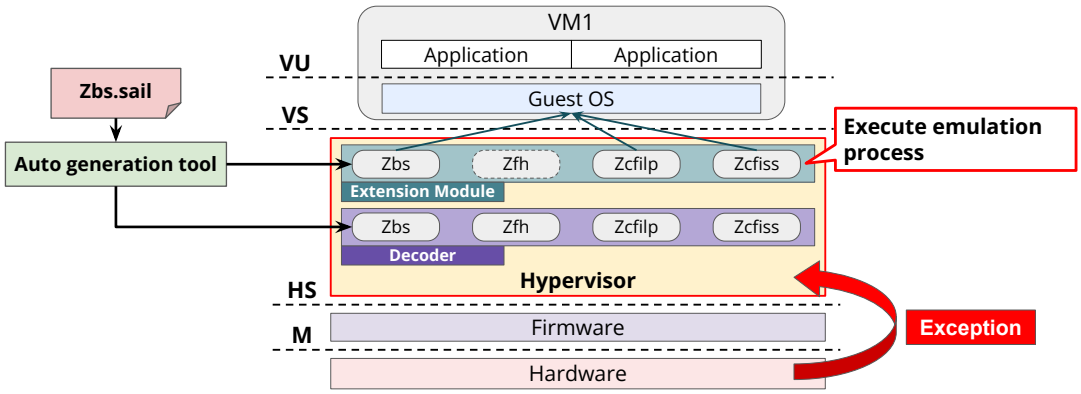

Figure 2. High-level architecture of the proposed emulation framework. The offline toolchain automatically generates highly reliable decoders and module templates from formal Sail specifications. The runtime Type-1 hypervisor sits between the physical M-mode hardware and the VS-mode guest software, intercepting and routing traps to the corresponding auto-generated extension modules.

3.2 Trap and Emulation Mechanism

The core performance advantage of our system relies on the efficient utilization of the RISC-V hypervisor extension's exception delegation mechanism. By configuring the machine delegation registers, we ensure that specific exceptions trigger a context switch to HS-mode, allowing standard instructions to execute natively. The new architectural behaviors introduced by RISC-V extensions can be broadly categorized into three types: new instructions, new CSRs, and new exception codes. By trapping and emulating these behaviors in the hypervisor, we can present the illusion to the guest software that the underlying hardware fully supports the extension. The following sections detail the emulation methodology for each category.

3.2.1 Instruction Emulation. Figure 3 illustrates the emulation flow for newly defined instructions. When software targeting an unimplemented extension runs on legacy CPU hardware, executing an unsupported instruction triggers an exception. In Figure 3, the highlighted `bset` instruction exemplifies this scenario. In the RISC-V architecture, attempting to execute an unknown instruction raises an Illegal Instruction exception; the `scause` register is set to exception code 2, and the faulting instruction itself is captured in the `stval` register [3].

The hypervisor passes the value in `stval` to the decoder to identify the instruction type. It then mimics the instruction's exact architectural behavior by directly reading or writing guest memory and updating the saved general-purpose registers within the trap context. If the emulated semantics dictate that an exception should occur, the hypervisor injects the corresponding exception into the guest (detailed in Section 3.2.3). Finally, the hypervisor advances the guest's program counter (`sepc`) to the subsequent instruction (e.g., `addi sp, sp, -60`) and resumes Virtual User (VU) mode

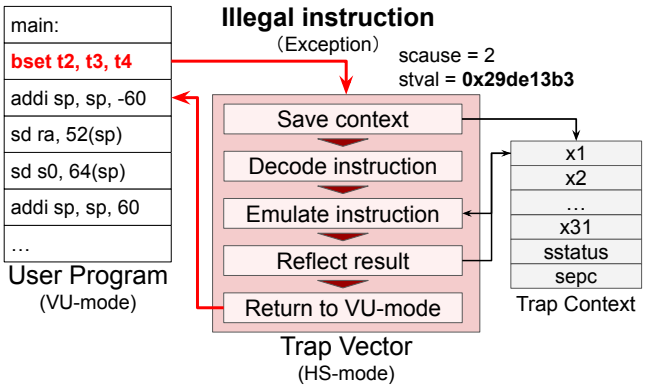

Figure 3. Overview of instruction emulation via exception trapping. The hypervisor intercepts the illegal instruction, decodes it, mimics its architectural behavior, and resumes guest execution.

execution via the `sret` instruction. From the guest's perspective, the instruction appears to have executed natively.

To successfully emulate an instruction using this hypervisor-based approach, the instruction must satisfy three fundamental requirements:

- **Guaranteed Exception Generation:** The execution of the instruction must invariably trigger an exception. Consequently, modifications to architectural behavior that the hardware processes transparently without raising an exception cannot be emulated. Examples include Hint instructions, which the hardware treats as NOPs, or WPRI (Writes Preserve Values, Reads Ignore Values) fields in CSRs, which silently ignore writes [2].
- **Privilege Level Constraints:** The target instruction must be executable at a privilege level lower than the hypervisor's HS-mode. Instructions that mandate

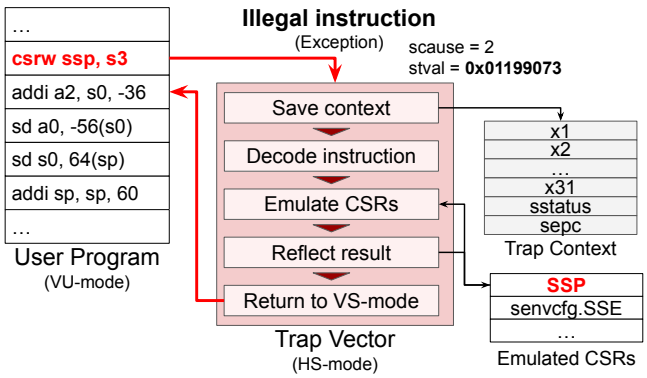

Figure 4. Overview of CSR emulation. The hypervisor decodes the trapped CSR instruction and maintains the virtualized CSR state in its own memory space.

Machine mode (M-mode) execution, or accesses to M-mode-specific CSRs (e.g., `mstatus`, `misa`), cannot be natively emulated by the hypervisor due to insufficient privileges. Similarly, the hypervisor cannot emulate the H extension itself, as the system fundamentally relies on these hardware virtualization features to operate.

- **Hardware Independence:** Instructions that rely on specialized, low-level hardware mechanisms cannot be fully replicated via simple memory and register manipulations. For instance, atomic instructions (A extension) that require physical bus locks or exclusive access mechanisms, and cache management instructions (e.g., Zicbom) that mandate physical cache line operations, are outside the scope of reliable software emulation.

3.2.2 CSR Emulation.

When a RISC-V extension introduces new Control and Status Registers (CSRs), it typically follows one of two patterns: defining an entirely new CSR, or adding new fields to an existing CSR. The former reliably triggers an exception upon access, making it straightforward to emulate. The latter, however, fails to generate an exception natively and requires a specialized workaround.

Emulating Newly Added CSRs. Attempting to execute an instruction that accesses an unknown CSR inherently raises an Illegal Instruction exception [3]. Figure 4 outlines the emulation process for these new CSRs.

While the faulting instruction (e.g., `csrw ssp, s3`) is stored in `stval`, the specific CSR address is not directly provided in a separate trap register. The hypervisor must utilize the decoder to parse the instruction and extract both the target CSR address and the involved general-purpose registers (e.g., `ssp` and `s3` in Figure 4). Values read from the virtual CSR are written to the saved Trap Context, while values intended to be written to the CSR are stored in a dedicated memory region managed by the hypervisor (Emulated CSRs).

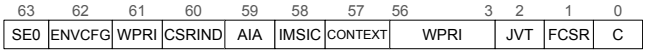

Figure 5. Field layout of the `hstateen0` register utilized to intercept accesses to newly defined fields in existing CSRs.

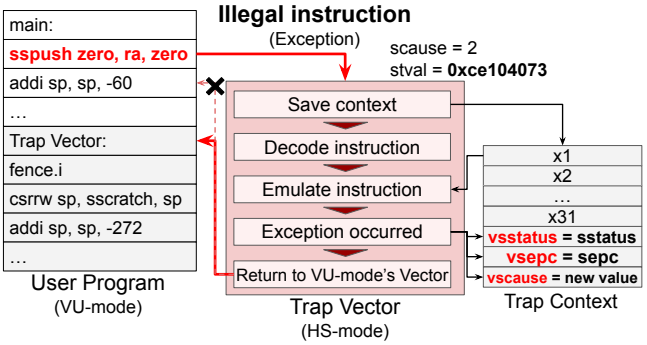

Figure 6. Simulating exception injection by modifying guest trap registers and altering the hypervisor return address.

Emulating New Fields in Existing CSRs. In the RISC-V specification, undefined fields within existing CSRs are designated as WPRI [3]. Hardware implementations silently ignore writes to WPRI fields and return constant values upon reads, failing to trigger the traps necessary for hypervisor intervention.

To overcome this limitation, our research leverages the `Ssstateen` (State Enable) extension. The `Ssstateen` extension allows software to explicitly enable or disable access to specific extension states, primarily to prevent inadvertent information leakage during context switches [3]. We repurpose this mechanism to intentionally prohibit access to certain registers from VS-mode and VU-mode. By clearing the corresponding bit in the hypervisor's `hstateen0` register (illustrated in Figure 5), we force the hardware to raise an exception.

For example, to trap accesses to the `senvcfg` register, the hypervisor sets the `ENVCFG` field (bit 62) of `hstateen0` to 0. Once the forced exception occurs, the hypervisor can intercept the access and emulate the new CSR fields using the same flow established for entirely new CSRs.

3.2.3 Exception Emulation.

During the emulation of an instruction or a CSR access, the hypervisor may need to raise an exception as dictated by the extension's semantics. The methodology for simulating this exception injection is depicted in Figure 6.

Although the hypervisor cannot force the underlying hardware to spontaneously generate a new exception on behalf of the guest, it can perfectly simulate the architectural outcome. Instead of advancing the return address to the next instruction (the red dotted arrow in Figure 6), the hypervisor redirects the guest's execution flow by setting `sepc` to the guest's trap vector address (defined in the `vstvec` register).

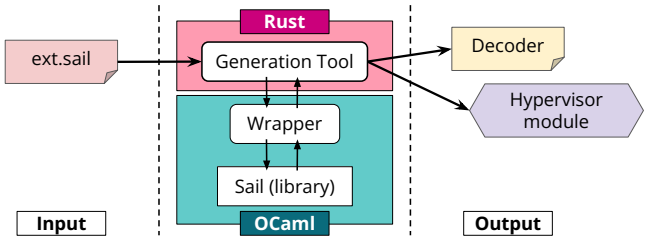

Figure 7. The Sail-driven auto-generation framework. The framework utilizes the official Sail compiler via FFI to parse specifications, generates hypervisor module templates, constructs a decision tree for decoding, and outputs highly reliable Rust decoders and unit tests.

Furthermore, if the extension defines a novel exception cause, the hypervisor populates the guest's virtual cause register (`vscause` within the Trap Context) with the new exception number. By carefully staging the guest's CSRs and redirecting the program counter before executing `sret`, the hypervisor flawlessly simulates the hardware exception delivery process to the guest OS.

3.3 Sail-Driven Automated Generation

A fundamental challenge in developing software emulators is ensuring that their behavior perfectly matches the ISA specifications. Historically, developers have manually transcribed instruction encodings and semantics from human-readable PDF manuals into implementation languages such as C or Rust. This manual process is notoriously prone to subtle transcription errors, which can compromise the validity of the entire software testing environment.

To mitigate this risk and directly address the need for a reliable emulation environment with minimal manual effort, we integrated a novel auto-generation framework into our system. We leverage the official formal specifications of the RISC-V ISA written in Sail [4]. Sail is a formal semantics language that provides an unambiguous and rigorous foundation for these specifications. Figure 7 illustrates our auto-generation framework, which bridges the formal specifications and the practical Rust implementation through the following steps.

3.3.1 Formal Specifications and AST Parsing. The official RISC-V specifications are rigorously defined in `.sail` files. To ensure absolute correctness, our auto-generation framework does not rely on fragile textual parsing. Instead, it utilizes a Foreign Function Interface (FFI) to directly invoke the core functions of the official OCaml-based Sail compiler.

Through this wrapper library, the framework commands the Sail compiler to parse the specifications, perform strict type-checking, and transform the formal definitions into an Abstract Syntax Tree (AST). The framework then retrieves this thoroughly validated AST via JSON serialization. By extracting instruction unions (e.g., `union clause ast`) and CSR mappings (e.g., `csr_name_map`) programmatically from this verified AST, we establish a rigorous foundation for the emulator.

Listing 1. Auto-generated hypervisor module template. The framework enumerates all required instructions and utilizes `todo!()` macros, directing developers exactly where to implement the semantics.

```
impl EmulateExtension for Zbs {
    fn instruction(&mut self, inst: &Instruction) {
        let mut context = unsafe { /* ... */ };
        match inst.opc {
            OpcodeKind::Zbs(ZbsOpcode::BSET) => {
                /* Developer implements semantics here */
            }
            OpcodeKind::Zbs(ZbsOpcode::BCLR) => todo!(),
            OpcodeKind::Zbs(ZbsOpcode::BEXT) => todo!(),
            // ... automatically enumerated instructions
            _ => unreachable!(),
        }
    }
}
```

3.3.2 Hypervisor Module Templating. Based on the extracted AST, the framework first accelerates the development process by generating the boilerplate code for the hypervisor modules. It scaffolds the Rust structures (`struct`) and automatically implements the required `trait` interfaces that allow the module to interact with the hypervisor's context management.

Crucially, because generating the exact semantic execution logic is challenging, the framework inserts Rust's `todo!()` macros at the execution points of each defined instruction (Listing 1). Developers simply replace these macros with the actual architectural behaviors. This systematic division of labor guarantees that developers will not forget to implement any instructions or make mistakes in the dispatch logic. It significantly narrows the Trusted Computing Base (TCB) associated with human errors.

3.3.3 Deterministic Decoder Generation. Once the module templates are in place, the hypervisor needs a mechanism to identify which instruction triggered the trap. Decoding RISC-V instructions is a multi-step process that involves masking and shifting specific bit-fields (e.g., `opcode`, `funct3`, `funct7`) to determine the exact instruction type. Doing this manually for dozens of new instructions in an extension is highly error-prone.

To automate this, our framework analyzes the AST to extract bit-patterns and operand layouts, dynamically constructing a deterministic decision tree. The algorithm operates as follows:

1. **Initial Grouping:** It groups all instructions within an extension based on their primary 7-bit `opcode` (bits 0–6).
2. **Recursive Branching:** For subsets containing multiple instructions, it selects subsequent sub-fields. To optimize the decision tree, fields with smaller bit-widths are prioritized for branching.

```rust
pub fn parse_opcode(inst: u32) -> Result<ZbbOpcode, DecodingError> {
    let op_6_0 = inst.slice(6, 0);
    let op_14_12 = inst.slice(14, 12);
    let op_31_25 = inst.slice(31, 25);
    match op_6_0 {
        // ...
        0b110011 => match op_14_12 {
            0b001 => Ok(ZbbOpcode::ROL),
            0b100 => match op_31_25 {
                0b00101 => Ok(ZbbOpcode::MIN),
                0b100000 => Ok(ZbbOpcode::XNOR),
                _ => Err(DecodingError::InvalidOpcode),
            },
            // ...
            _ => Err(DecodingError::InvalidOpcode),
        },
        _ => Err(DecodingError::InvalidOpcode),
    }
}
```

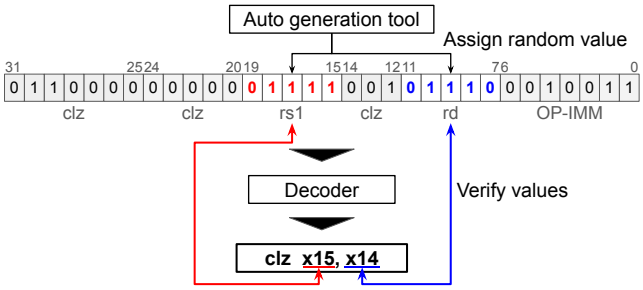

Figure 8. Overview of the automated unit test generation. Random operands are synthesized into a binary instruction, which is then parsed by the auto-generated decoder to verify correctness.

3. **Resolution:** This process repeats recursively until every instruction is uniquely identified. If an instruction lacks a specific field utilized by others in the subset, a wildcard match is inserted.

The framework then traverses this decision tree to emit deeply nested, highly optimized `match` statements in Rust (Listing **??**). To ensure memory safety and robustness, it automatically inserts fallback arms (e.g., `_ => Err(DecodingError::InvalidOpcode)`) for any unrecognized bit patterns. This generated Rust code acts as a precise, zero-cost instruction decoder, eliminating manual transcription faults.

label,

3.3.4 Automated Unit Test Generation.

While the auto-generation framework drastically reduces manual coding, the generation tool itself is written by humans and may contain flaws. Furthermore, Sail's rich language features mean that parsing corner cases could introduce bugs. To ensure the reliability of the generated decoders, rigorous testing is indispensable.

As depicted in Figure 8, our framework resolves this by automating the generation of unit tests directly from the Sail specifications.

Because the framework precisely understands the operand bit-widths (e.g., 5 bits for `rd` and `rs1`), it generates unit tests that inject randomized valid operand values into the correct bit positions to construct a 32-bit binary instruction. The test then passes this synthesized binary to the auto-generated decoder and asserts that the resulting opcode and extracted operands perfectly match the expected values.

By integrating these auto-generated tests into the continuous integration (CI) pipeline, we can systematically verify that the decoding logic remains strictly compliant with the RISC-V ISA. This automated verification step provides immediate detection of any logic flaws, further reducing the manual testing burden and enhancing the overall trustworthiness of the emulation framework.

Table 3. Hardware specifications of the Milk-V Megrez [7].

Feature	Description
ISA	RV64GC_Zba_Zbb_Sscofpmf
Privilege Modes	M, U, HS, VU, VS
L1 Instruction Cache	32 KiB 4-way
L1 Data Cache	32 KiB 4-way
Private L2 Cache	256 KiB 8-way with 2 banks
L3 Cache	4 MiB 16-way with 4 banks
MMU	Sv48 virtual memory support

4 Evaluation

In this section, we evaluate the effectiveness of our proposed system from three perspectives: practicality, robustness, and development efficiency. Specifically, we conduct quantitative evaluations on real hardware focusing on the following metrics:

- Source Lines of Code (SLoC) and Memory Footprint (Resource efficiency)
- Hypervisor setup and OS boot time (Practical system performance)
- Interrupt latency (Real-time responsiveness)
- Instruction counts required for emulation (Processing overhead)

Furthermore, we compare our system against QEMU, a state-of-the-art full-system emulator, to clarify the positioning and advantages of our approach.

4.1 Experimental Setup

All hardware evaluations were conducted on a Milk-V Megrez board equipped with the EIC7700X SoC, which natively supports the RISC-V hypervisor extension (Table 3). For the software environment (Table 4), we modified the vendor-provided U-Boot to load our hypervisor instead of the OS kernel directly, passing hardware information (e.g., `hartid`) as boot arguments. The Linux kernel was built using the exact configuration of the vendor's proven image to ensure a fair comparison.

Table 4. Software environment used for the evaluation.

Component	Version	Base Repository
U-Boot	2024.01	milkv-megrez/rockos-u-boot
OpenSBI	v1.5	milkv-megrez/rockos-opensbi
Linux	6.6.87	rockos-riscv/rockos-kernel

Table 5. SLoC of the hypervisor and related crates.

Component	Files	Lines
Hypervisor Binary	11	1,734
Core Library	32	5,246
Zbs Module	1	**109**

4.2 SLoC and Memory Footprint

Our system consists of a core library, the hypervisor binary, and extension modules. Table 5 shows the SLoC for these components. The entire Type-1 hypervisor is remarkably lightweight, with the core library and main binary comprising roughly 5,200 and 1,700 lines of Rust code, respectively. Notably, the `Zbs` extension module requires only 109 lines (including document comments) and is packaged as an entirely independent crate, completely decoupled from the hypervisor's core codebase. This minimal, isolated footprint is a direct benefit of our auto-generation framework, which drastically reduces the code developers must write manually.

Regarding memory footprint, the `.text` section occupies merely 84 KiB, comfortably fitting within the 256 KiB L2 cache. This minimizes cache miss penalties during hypervisor execution. As shown in the binary bloat analysis, the `Zbs` emulation logic accounts for only 1.1% of the `.text` section. This demonstrates excellent scalability; adding new extension modules incurs negligible size increases.

4.3 Boot Time and Interrupt Latency

4.3.1 Setup and Boot Time. We measured the hypervisor setup time utilizing the hardware `mtime` register (1 MHz frequency). The setup phase—from hardware hand-off to guest OS execution—takes approximately 65 ms. This delay is virtually imperceptible in the context of the total system power-on time.

To evaluate OS boot overhead, we compared Linux boot times natively versus on our hypervisor (Table 6). The hypervisor introduces a minor overhead, increasing the boot time by approximately 7% in kernel space and 6% overall. This is primarily attributed to interrupt virtualization, which remains well within acceptable margins for development and validation purposes.

Table 6. Comparison of Linux boot times.

Environment	Kernel	User Space	Total
Native	12.189 s	19.080 s	31.270 s
Proposed System	13.060 s	20.198 s	33.258 s
Overhead Ratio	1.07	1.06	1.06

4.3.2 Interrupt Latency. Real-time responsiveness is critical for embedded systems. We evaluated the latency introduced by trapping physical interrupts in the hypervisor and injecting them as virtual interrupts into the guest.

Using the `time` register and `instret` performance counter, we analyzed timer interrupts injected every 10 ms. The processing required 131 instructions, resulting in an average latency of 1 μs. For external interrupts (driven by a 24 MHz clock), the hypervisor executed 178 instructions, yielding a latency of 0.5 μs.[1] In both cases, the sub-microsecond latency proves that our system maintains strict responsiveness suitable for embedded Linux workloads.

4.4 Emulation Overhead

We quantified the instruction overhead for emulating a single `bset` instruction from the Zbs extension. When the decoder cache is hit, the entire emulation flow—from exception entry, module invocation, semantic execution, to guest resumption—consumes 297 instructions. Approximately 50% of this overhead stems from the dynamic module invocation, a trade-off made for strict modularity and flexibility.

Similarly, emulating a write to a newly defined CSR (`ssp`) requires 288 instructions (with cache hit). While hundreds of times more costly than native execution, these operations (e.g., driver initialization or specific algorithmic accelerations) occur infrequently enough that their impact on macroscopic system performance is limited.

4.5 Comparison with QEMU

To evaluate system-level performance (RQ1 and RQ3), we compared our approach against QEMU using the CoreMark benchmark [11, 14]. Table 7 shows that while QEMU (Full Emulation) operates at only 8.5% of native speed, our proposed system maintains 99.8% of native performance. This overwhelming advantage stems from allowing non-target instructions to execute directly on the hardware.

We further analyzed the emulation of 1,000,000 consecutive `bset` instructions. Due to its JIT-compilation and Translation Block (TB) caching, QEMU executes tight loops exceptionally fast (865 μs). Our system, which traps on every

[1] Although external interrupts require more instructions than timer interrupts, the resulting latency is lower. This discrepancy can be attributed to the significantly higher clock resolution of the external timer (24 MHz vs. 1 MHz) and variations in the execution cycles required for different instruction types.

Table 7. CoreMark benchmark scores across environments.

Environment	Score (Iter/Sec)	Ratio (Native=1.0)
Native	10,686.044	1.000
QEMU (User Mode)	1,432.357	0.134
QEMU (Full Emulation)	903.120	0.085
Proposed System	10,661.549	**0.998**

Table 8. Execution time (μs) for 1,000,000 instructions with varying ratios of emulated `bset` instructions.

Environment	1%	0.1%	0.01%	0%
QEMU (Default)	1,219	1,168	1,231	1,302
QEMU (singlestep)	337,905	359,637	374,629	376,380
Proposed System	4,912	**885**	**358**	357

instruction, took 463,227 μs. However, when forcing QEMU to execute instruction-by-instruction (`singlestep`), it took 480,188 μs. This confirms that our trap-and-emulate overhead is fundamentally on par with QEMU's base software overhead.

Crucially, realistic workloads consist of a mix of native and emulated instructions. Table 8 demonstrates the total execution time for workloads containing varying percentages of the `bset` instruction.

When emulated instructions account for 1%, QEMU is faster. However, at a ratio of 0.1%, our system becomes 1.3 times faster than QEMU, and at 0.01%, it is 3.4 times faster. In practical OS operations, hardware-supported RV64GC instructions heavily dominate the execution flow. Therefore, our system provides a significantly faster and more accurate validation environment for real-world scenarios. Additionally, unlike QEMU, which requires extensive custom device models, our hypervisor allows developers to use unmodified SoC vendor kernels and drivers, dramatically reducing environment setup costs.

4.6 Robustness and Productivity via Automation

Finally, we evaluate how our Sail-driven auto-generation contributes to system robustness (RQ2). Implementing an extension manually requires parsing PDF specifications, writing complex decoders, scaffolding Rust traits, and implementing semantics. This process is inherently vulnerable to human errors, such as omitted instructions or flawed bit-masking in decoders.

Our toolchain automates the parsing, decoder generation, and module scaffolding directly from formal specifications. This guarantees parity between the ISA specification and the Rust implementation, eliminating transcription faults by design. Quantitatively, out of the 109 lines in the `Zbs` module, 81 lines were entirely auto-generated. Developers only needed to implement 28 lines of isolated semantic logic. This drastically reduces development effort and narrows the potential surface area for bugs, establishing a highly trustworthy foundation for embedded software validation.

5 Conclusion

The rapid expansion of RISC-V extension specifications has created a significant gap between standardization and physical hardware availability. This lag severely bottlenecks the early development, validation, and testing of software stacks. In this paper, we proposed a lightweight Type-1 hypervisor that leverages the RISC-V hypervisor extension to emulate the semantics of unimplemented instructions, CSRs, and exceptions. By executing the guest software natively on the hardware and trapping only the unallocated operations, our system maintains 99.8% of native performance for non-target workloads and restricts interrupt latency to under 1 μs. Furthermore, our evaluation demonstrates that it outperforms a widely-used full-system emulator, QEMU, in realistic workloads where emulated instructions account for 0.1% or less of the total execution.

To guarantee the correctness of this validation environment and minimize manual effort, we introduced a novel auto-generation framework driven by Sail, the formal semantics of the RISC-V ISA. By automatically deriving precise instruction decoders and hypervisor module templates directly from the AST, we eliminated manual transcription errors and drastically reduced the necessary source lines of code, establishing a highly trustworthy and highly productive foundation for embedded software testing.

For future work, we plan to further advance our auto-generation framework by directly parsing the Sail AST to automatically synthesize the core execution logic of the instruction emulation itself, thereby further minimizing manual intervention. Additionally, we aim to integrate the Advanced Interrupt Architecture (AIA) and evaluate our emulator-verified Input-Output Memory Management Units (IOMMU) support on physical hardware. While silicon supporting these features was previously unavailable, recently announced commercial chips like the SpacemiT K3 series make this viable [26]. Supporting AIA will allow the direct injection of virtual interrupts into the guest OS, which will significantly mitigate hypervisor overhead. Simultaneously, leveraging physical IOMMU support will enable more flexible and secure memory management for direct device assignment. Ultimately, these enhancements will further solidify our framework as a practical and highly capable validation environment, breaking the vicious cycle between hardware availability and software readiness, and thereby accelerating the overall development lifecycle for emerging RISC-V extensions.

References

[1] ahadali5000, Alasdair Armstrong, Alexander Richardson, Aril Computer Corp., Scott Johnson, Ben Marshall, Bicheng Yang, Muhammad Bilal Sakhawat, Brian Campbell, Chris Casinghino, Christopher Pulte, Codasip, Tim Hutt, Martin Berger, Ben Fletcher, dylux, eroom1966, Google LLC, Hesham Almatary, Jan Henrik Weinstock, Jessica Clarke, Jon French, Jordan Carlin, Michael Sammler, Microsoft, Robert Norton-Wright, Nathaniel Wesley Filardo, Paul A. Clarke, Peter Rugg, Peter Sewell, Philipp Tomsich, Prashanth Mundkur, Rafael Sene, Rishiyur S. Nikhil, Shaked Flur, Thibaut Pérami, Thomas Bauereiss, VRULL GmbH, William McSpaddden, and Xinlai Wan. 2025. *Riscv/Sail-Riscv: Sail RISC-V Model.* https://github.com/riscv/sail-riscv

[2] Waterman Andrew, Lee Yunsup, Patterson David, and Avižienis Rimas. 2024. *The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture.* https://github.com/riscv/riscv-isa-manual/releases/download/20240411/priv-isa-asciidoc.pdf

[3] Waterman Andrew, Lee Yunsup, Avižienis Rimas, Patterson David, and Asanović Krste. 2024. *The RISC-V Instruction Set Manual Volume II: Privileged Architecture.* https://github.com/riscv/riscv-isa-manual/releases/download/20240411/priv-isa-asciidoc.pdf

[4] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for Armv8-a, RISC-V, and CHERI-MIPS. 3, Article 71 (2019). Issue POPL. doi:10.1145/3290384

[5] Krste Asanovic, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, et al. 2016. The Rocket Chip Generator. 4 (2016).

[6] Jonathan Behrens, Cel Skeggs, Samuel Ortiz, and Frans Kaashoek. 2025. *Mit-Pdos/RVirt.* https://github.com/mit-pdos/RVirt

[7] Beijing ESWIN Computing Technology Co., Ltd. 2025. EIC7700X SoC Technical Reference Manual. https://github.com/eswincomputing/EIC7700X-SoC-Technical-Reference-Manual

[8] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference* (Anaheim, CA) *(ATEC '05)*. USENIX Association, USA, 41.

[9] Andrea Bocco, Yves Durand, and Florent De Dinechin. 2019. SMURF: Scalar Multiple-precision Unum RISC-V Floating-point Accelerator for Scientific Computing. In *Proceedings of the Conference for Next Generation Arithmetic 2019* (New York, NY, USA, 2019-03-13) *(CoNGA'19)*. Association for Computing Machinery, 1–8. doi:10.1145/3316279.3316280

[10] Carl Friedrich Bolz-Tereick, Luke Panayi, Ferdia McKeogh, Tom Spink, and Martin Berger. 2025. *Pydrofoil: Accelerating Sail-based Instruction Set Simulators.* arXiv:2503.04389 [cs] doi:10.48550/arXiv.2503.04389

[11] Celio Christopher, Karandikar Sagar, Zhao Jerry, and Gonzalez Abraham. 2025. *Riscv-Boom/Riscv-Coremark.* https://github.com/riscv-boom/riscv-coremark

[12] Lowie Deferme and Dominique Devriese. 2024. RISC-V Hypervisor extension formalization in Sail. (2024).

[13] dramforever. 2024. *Dramforever/Opensbi-h: WIP: A Fork of OpenSBI, with Software-Emulated Hypervisor Extension Support.* https://github.com/dramforever/opensbi-h

[14] EEMBC. 2025. *CPU Benchmark – MCU Benchmark – CoreMark – EEMBC Embedded Microprocessor Benchmark Consortium.* https://www.eembc.org/coremark/index.php

[15] Gao Han. 2025. *rockos-riscv/rockos-kernel at cd464ac64b38c747524d7407570720249680b23d.* https://github.com/rockos-riscv/rockos-kernel/tree/cd464ac64b38c747524d7407570720249680b23d

[16] Yunsup Lee, Andrew Waterman, Rimas Avizienis, Henry Cook, Chen Sun, Vladimir Stojanović, and Krste Asanović. 2014. A 45nm 1.3GHz 16.7 Double-Precision GFLOPS/W RISC-V Processor with Vector Accelerators. In *ESSCIRC 2014 - 40th European Solid State Circuits Conference (ESSCIRC)* (2014-09). 199–202. doi:10.1109/ESSCIRC.2014.6942056

[17] José Martins, Adriano Tavares, Marco Solieri, Marko Bertogna, and Sandro Pinto. 2020. Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems. (2020). doi:10.4230/OASIcs.NG-RES.2020.3

[18] Pascal Pieper, Vladimir Herdt, and Rolf Drechsler. 2022. Advanced Embedded System Modeling and Simulation in an Open Source RISC-V Virtual Prototype. *Journal of Low Power Electronics and Applications* 12, 4 (2022). doi:10.3390/jlpea12040052

[19] RISC-V International. 2024. *Exchange – RISC-V International.* https://riscv.org/exchange/?_sft_exchange_category=board,hardware

[20] RISC-V International. 2024. *RISC-V International – RISC-V: The Open Standard RISC Instruction Set Architecture.* https://riscv.org/

[21] RISC-V International. 2024. *RISC-V Profiles Version 1.0.* https://github.com/riscv/riscv-profiles/releases/download/v1.0/profiles.pdf

[22] RISC-V International. 2025. *Riscv/Riscv-Profiles.* https://github.com/riscv/riscv-profiles

[23] Jeff Scheel. 2025. *Ratified Extensions - Home - RISC-V Tech Hub.* https://riscv.atlassian.net/wiki/spaces/HOME/pages/16154732/Ratified+Extensions

[24] Colin Schmidt and Adam Izraelevitz. 2015. A Fast Parameterized SHA3 Accelerator. http://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-204.html

[25] Shenzhen MilkV Technology Co., Ltd. 2025. *Milk-V Megrez | RISC-V AI PC.* https://milkv.io/megrez

[26] SPACEMIT. 2026. Introduction to K3 Series Chips. https://cdn.sipeed.com/public/k3/Introduction%20to%20K3%20Series%20Chips.pdf

[27] SpinalHDL. 2025. *SpinalHDL/VexRiscv.* https://github.com/SpinalHDL/VexRiscv

[28] Bruno Sá, José Martins, and Sandro Pinto. 2021. A First Look at RISC-V Virtualization from an Embedded Systems Perspective. (2021). doi:10.48550/arXiv.2103.14951

[29] Xiang W and Gao Han. 2025. *Rockos-Opensbi/Lib/Sbi/Sbi_emulate_csr.c at Rockos-Dev · Rockos-Riscv/Rockos-Opensbi.* https://github.com/rockos-riscv/rockos-opensbi/blob/rockos-dev/lib/sbi/sbi_emulate_csr.c

[30] Florian Zaruba and Luca Benini. 2019. *The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-Nm FDSOI Technology.* doi:10.1109/TVLSI.2019.2926114